

Distribuirani sistemi

(VIII semestar 2+2+1)

Predavanja: Emina Milovanović, Natalija Stojanović

Rač. vežbe: Aleksandar Veljanovski, Aleksandra Stojnev,
Vukadin Drašković

Lab. vežbe: Aleksandar Veljanovski, Aleksandra Stojnev,
Vukadin Drašković

Web stranica predmeta: cs.elfak.ni.ac.rs/nastava/

Način polaganja i literatura

Način polaganja:

- | | |
|--|---------------|
| 1. Lab. Vežbe
(nakon svake oblasti, 4 lab. vežbe) | : 0 – 20 |
| 2. I kolokvijum | : 0 – 40(>20) |
| 3. II kolokvijum | : 0 – 40(>20) |
| 4. Pismeni | : 0 – 40 |
| 5. Usmeni | : 0 – 40 |

1. Konačna ocena = 1+2+3
2. Konačna ocena = 1+4+5

Literatura:

1. Andrew S. Tanenbaum, Maarten Van Steen, Distributed Systems, Fourth edition, 2023.
2. George Coulouris, Jean Dollimore, Tim Kondberg, Distributed Systems: Concepts and design, Fifth Edition, Addison Wesley, 2012.

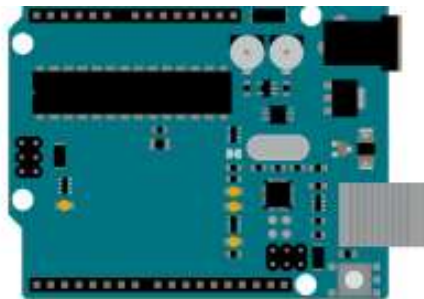
Uvod

- * Od 1945. do sredine 1980-ih računari su bili veoma skupi i glomazni.
- * Sredinom 1980-ih napredak u tehnologiji je doprineo promeni situacije.
- * Dve stvari su odigrale ključnu ulogu:
 - Pojava mikroprocesora
 - Pojava mikroprocesora omogućila je gradnju RS po niskoj ceni uz istovremeno poboljšanje performansi
 - računari su postali manji, jeftiniji, brži, efikasniji sa stanovišta potrošnje energije
 - Mogućnost projektovanja veoma brzih računarskih mreža
 - Veoma brze mreže omogućile su razmenu poruka između dva računara brzinom od 10Mbit/s do 1Gbit/s.
 - Kao rezultat ovih tehnologija postalo je ne samo moguće, već i isplativo, povezati veliki broj računara korišćenjem veoma brzih mreža.

Uvod

Računari su postali:

- Brži
- Jeftiniji
- Efikasniji sa stanovišta potrošnje
- Manjih dimenzija



Arduino UNO: weight=25 g, width=53.4 mm, length=68.6 mm

- 1974: Intel 8080
 - 2 MHz, 6K transistors
- 2004: Intel P4 Prescott
 - 3.6 GHz, 125 million transistors
- 2011: Intel 10-core Xeon Westmere-EX (**multicore CPUs**)
 - 3.33 GHz, 2.6 billion transistors
- 2019: NVIDIA Turing GPU
 - 14.2 TFLOPS of peak single precision (FP32) performance

Memorija

memorije
16,000x jeftije
4,000x veći kapacitet

godina	Cena \$/MB	Tipična veličina
1977	\$32,000	16K
1987	\$250	640K-2MB
1997	\$2	64MB-256MB
2007	\$0.06	512MB-2GB+

Brzina prenosa

35,000x
brzina prenosa

* brzine prenosa u LAN

- 1980: Original Ethernet: 2.94 Mbps
- 1985: thick Ethernet: 10 Mbps
- 1991: 10BaseT - twisted pair: 10 Mbps
- 1995: 100 Mbps Ethernet
- 1998: 1 Gbps (Gigabit) Ethernet
- 1999: 802.11b (wireless Ethernet) standardized
- 2001: 10 Gbps
- 2005: 100 Gbps (preko optičkog linka)

Date	Hosts
Dec 1979	188
July 1989	130,000
July 1999	56,218,000
July 2001	125,888,197
July 2003	171,638,297
July 2005	353,284,187
July 2006	439,286,364
July 2007	489,774,269
July 2008	570,937,778
July 2009	681,064,561
July 2010	768,913,036
July 2011	849,869,781
July 2012	908,585,739
July 2013	996,230,757
July 2014	1,028,544,414
July 2015	1,033,836,245

Date	Hosts	Web servers
July 1993	1,776,000	130
July 1995	6,642,000	23,500
July 1997	19,540,000	1,203,096
July 1999	56,218,000	6,598,697
July 2001	125,888,197	31,299,592
July 2003	212,570,000	42,298,371
July 2005	353,284,187	67,571,581
July 2006	439,286,364	88,166,395
July 2007	489,774,269	125,626,329
July 2008	570,937,778	175,480,931
July 2009	681,064,561	239,611,111
July 2010	768,913,036	205,714,253
July 2011	849,869,781	357,292,065
July 2012	908,585,739	665,916,461
July 2013	996,230,757	698,823,509
July 2014	1,028,544,414	996,106,380
July 2015	1,033,836,245	849,602,745

Source: Internet Systems Consortium

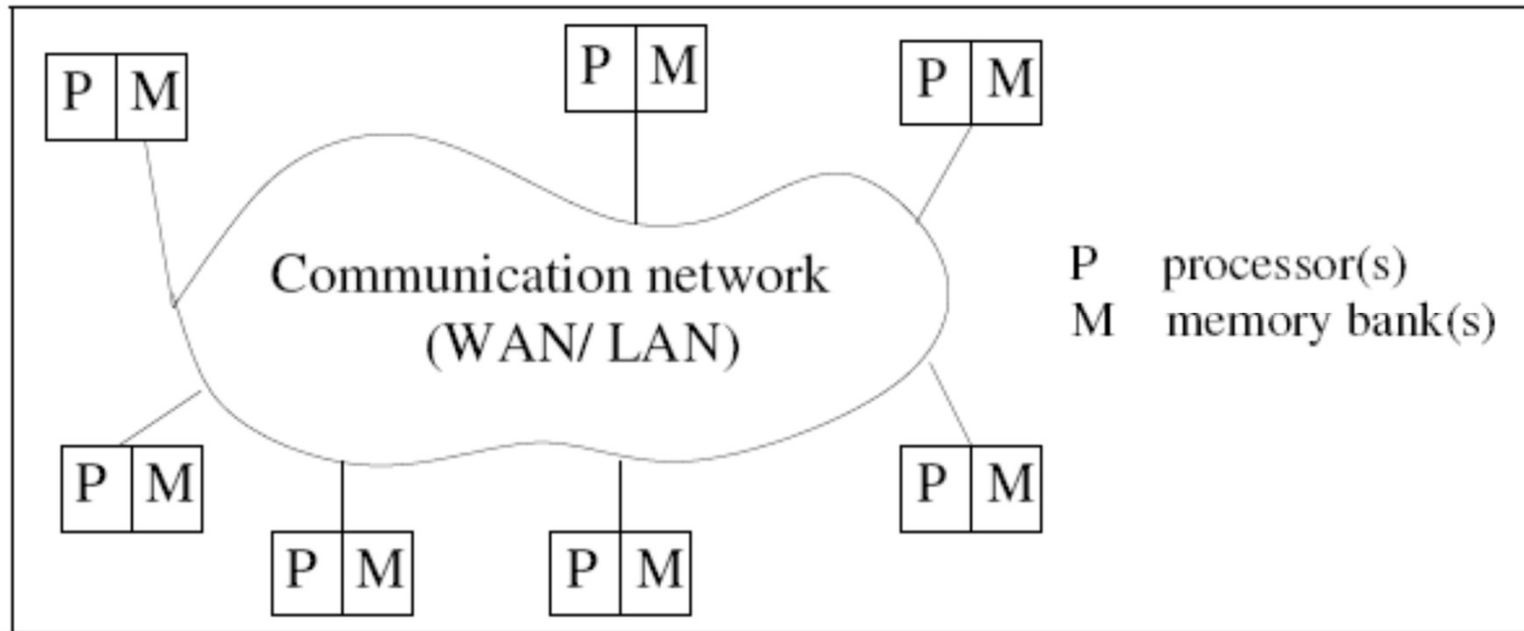
Šta je Distribuirani sistem?

“A collection of independent computers that appears to its users as a single coherent system”
-- A.S. Tanenbaum, M. van Steen

“One in which hardware or software components located at networked computers communicate and coordinate their actions only by passing messages”
-- G. Coulouris, J. Dollimore, T. Kindberg

“One on which I cannot get any work done because some machine I have never heard of has crashed”
-- L. Lamport

Model distribuiranog sistema



DS povezuje procesore pomoću komunikacione podmreže

Prednosti DS nad centralizovanim

* Ekonomski

- DS imaju bolji odnos cena/performanse od velikih (mainframe) računara.
 - Grace Hopper (poznati naučnik u oblasti CS): "In pioneer days they used oxen for heavy pulling, and when one ox couldn't budge a log, they didn't try to grow a larger ox. We shouldn't be trying for bigger computers, but for more systems of computers".
 - Umesto jednog mainframe računara koji košta desetine miliona dolara, ista moć obrade se može dobiti gradnjom klastera za desetak hiljada dolara

* Brzina:

- DS može imati veću ukupnu moć obrade nego mainframe.
 - Npr. Jedna velika baza podataka može biti razbijena na manje baze. Na taj način se vreme odziva može poboljšati
 - Distribirane baze mogu biti hijerarhijski organizovane (primer DNS).
 - Google pretraživač

* Pouzdanost:

- Ako imate centralizovani sistem i računar otkáže, aplikacija ne može da se izvršava
- Kod DS (ako je dobro projektovan) ako 15% mašina otkáže, aplikacija i dalje može da se izvršava, tj. sistem i dalje može ostati u funkciji sa samo 15% degradacije u performansama.
 - Npr. Google (hiljade čvorova može biti van funkcije u svakom trenutku, ali je usluga pretraživanja i dalje dostupna.
 - To je zato što je sistem dobro projektovan!

* Inkrementalni rast:

- sistem se može postupno povećavati dodavanjem novih računara.

* Deljenje resursa:

- Omogućeno je da više korisnika pristupa zajedničkim bazama podataka i periferijama.
 - Tipčan primer distribuirani fajl sistem
 - web

* Komunikacija:

- olakšana komunikacija između ljudi.

* Efikasno korišćenje resursa:

- omogućava da se opterećenje rasporedi na raspoložive računare na najefikasniji način

Mane

* Mnogo veća kompleksnost DS od centralizovanog

● Softver:

- teže je razviti softver za DS nego za centralizovane sisteme
 - Ima više aktivnih procesa, koji međusobno komuniciraju
 - » Debugiranje (u slučaju da postoji greška) može biti noćna mora
- Neke od kompleksnosti mogu biti skrivene ili pomoću programskog jezika (biblioteke, framework-a) koji se koristi za pisanje aplikacije, ili OS ali to samo pomera problem sa aplikacije na PJ i/ili OS – sistem ostaje kompleksan
 - Distribuiranu aplikaciju ne možete napisati ako PJ koji koristite ne podržava interprocesnu komunikaciju.
 - Isto važi i za OS

* Umrežavanje:

● Povezanost je ključna za funkcionisanje DS

- mreža može otkazati ili postati zagušena

* Bezbednosni problemi

● Laka dostupnost resursa, dovodi do problema bezbednosti

- Dosadašnje metode zaštite pružaju relativno nizak nivo bezbednosti
 - lozinke i druge osetljive informacije se često šalju kroz mrežu kao običan tekst (bez šifriranja)

Osnovne osobine DS

- * Heterogenost
- * Transparentnost
- * Otvorenost
- * Skalabilnost

Heterogenost

✱ DS je sastavljen od heterogenog skupa računara.

- Heterogenost se ogleda u sledećem

- Hardver računara – različit skup instrukcija, različita interna prezentacija podataka.
- Operativni sistemi – interfejs za razmenu poruka se razlikuje od OS do OS (npr. Berkely i Winsoc).
- Programski jezici – karakteri i strukture podataka se različito predstavljaju u različitim programskim jezicima, što može biti problem ako aplikacije treba međusobno da komuniciraju.
 - Implementacije od strane različitih projekata – sve dok se zajednički standardi ne usvoje, različite implementacije ne mogu međusobno da komuniciraju. (OSI model je jedan način za rešavanje problema heterogenosti).

Transparentnost

* Transparentnost znači skrivanje nečega

- Važan cilj kod projektovanja DS je skrivanje činjenice da su njegovi procesi i resursi fizički distribuirani.
 - DS koji je u stanju da sakrije ove činjenice naziva se transparentnim
- Transparentnost je važan zahtev pri realizaciji jedinstvenog pogleda na sistem (single system image) koje čine da se DS može koristiti sa istom lakoćom kao jednoprocesorski sistem.
- Npr. Web – dozvoljava pristup informacijama, a da pri tome ne moramo znati na kom računaru se one nalaze; dovoljno je samo kliknuti na link!

Tipovi transparentnosti

* Pristupna transparentnost –

- podacima i resursima se pristupa na jedinstven način, bez obzira da li se oni nalaze na udaljenom ili lokalnom računaru
 - Npr. Različiti OS mogu koristiti različite načine imenovanja fajlova.
 - Razlike u imenovanju fajlova i način manipulacije fajlovima mora biti skriven za korisnika i aplikaciju

* Lokacijska transparentnost –

- korisnik ne zna (i ne mora da zna) gde je resurs fizički lociran u sistemu.
 - Imenovanje i ovde igra važnu ulogu.
 - Lokaciona transparentnost se može postići dodeljivanjem logičkih imena resursima
 - Primer URL imena: `http://www.prenhall.com/index.htm` ne nagoveštava gde se nalazi glavni web server izdavačke kuće Prentice Hall
 - Korisnik sa bilo koje lokacije na isti način pristupa resursu

* Migraciona transparentnost-

- Resurs može promeniti lokaciju a da klijent to ne primeti.
 - resurs se može kretati a da pri tome ne dolazi do promene imena resursa.(npr. mobilni telefoni, pozivalac i pozvani nisu svesni kretanja onog drugog)

Tipovi transparentnosti (nast.)

* Transparentnost konkurencije

- Omogućava da više procesa jednovremeno koristi isti resurs
 - Npr. Istovremeni pristup istom fajl severu ili istoj tabeli u deljivoj bazi podataka.
- Važno je postići da korisnici ne primete da se isti resurs koristi jednovremeno
- Jednovremeni pristup deljivom resursu mora ostaviti resurs u konzistentnom stanju
- Konzistentnost se postiže mehanizmima zaključavanja i sinhronizacije tako da se pristup deljivom resursu serijalizuje.

Tipovi transparentnosti (nast.)

* Transparentnost replikacije (umnožavanja)

- Omogućava postojanje više kopija istog resursa u DS u cilju povećanja performansi i pouzdanosti
 - Kopija se postavlja bliže mestu odakle se obavlja pristup
 - Sve kopije imaju isto ime
 - To znači da ako DS podržava replikaciju da automatski podržava i lokacionu transparentnost!
- Korisnici ili aplikacija nisu svesni postojanja više kopija resursa.
- Primer: DFS

* Transparentnost za otkaze-

- korisnik ne zna da je resurs u kvaru (ovo je jedan od najtežih problema kod DS. Sistem koji je otporan na otkaze mora biti u stanju da izvrši realokaciju resursa na delove sistema koji korektno funkcionišu)
 - Ponekad je veoma teško razlučiti da li je neki server (usluga) nedostupan ili je mreža zagušena.

Tipovi transparentnosti (nast.)

* Transparentnost paralelizacije

- Paralelizacija procesa se izvršava transparentno za aplikativnog programera i korisnika aplikacije
- Korisnik nije svestan da i kako je aplikacija paralelizovana i gde se izvršavaju procesi
- Primer: Hadoop

Otvorenost

- * Otvoreni DS nudi komponente koje se lako mogu koristiti od strane drugih sistema ili integrisati u druge sisteme
- * Otvoreni DS dozvoljavaju dodavanje novih servisa i omogućavaju dostupnost servisima od strane različitih klijenata
 - Otvoreni DS je sistem koji nudi usluge (servise) shodno standardnim pravilima koja definišu sintaksu i semantiku usluga.
 - Kod DS usluge su definisane interfejsima.
 - Interfejs definiše kako se pristupa usluzi
 - Interfejsi se opisuju pomoću posebnog jezika (IDL – Interface definition Language)
 - Opis interfeisa sadrži definicije funkcija koje su dostupne zajedno sa tipom parametara, tipom rezultata i mogućim izuzecima (exceptions)
 - Ako je interfeis dobro definisan, on omogućava proizvoljnom procesu kome je usluga potrebna da pristupi usluzi

Skalabilnost

- * Skalabilnost (proširljivost) DS je jedan od najvažnijih projektantskih zadataka.
- * Skalabilnost je svojstvo (distribuiranog) sistema da održava adekvatan nivo performansi bez obzira na porast: broja korisnika i resursa, udaljenosti između računara, broja administrativnih domena
- * Skalabilnost se može posmatrati kroz tri dimenzije:
 - Skalabilnost u odnosu na broj korisnika i resursa
 - Skalabilnost u odnosu na geografsku udaljenost resursa i korisnika
 - Administrativna skalabilnost – sistemom se može lako upravljati čak i ako se prostire kroz više administrativnih domena.
- * Skalabilnost se ipak plaća performansama sistema!
 - Ako je potrebno podržati više korisnika ili resursa često dolazi do problema zbog centralizovanih usluga, podataka ili algoritama.
 - Mnoge usluge su centralizovane u smislu da su implementirane samo na jednom serveru u DS.
 - Server može postati usko grlo u sistemu sa porastom broja korisnika.
 - Replikacija je moguća, ali nekad je neophodno korišćenje samo jednog servera (npr. Server sa bankovnim računima građana - kopiranje servera na nekoliko lokacija bi stvorilo dodatne bezbednosne probleme)
 - Centralizovani podaci, kao i centralizovane usluge, mogu stvoriti probleme
 - Npr. Da DNS nije implementiran kao distribuirana baza podataka, Internet bi imao malo smisla, jer bi zahtevi za preslikavanje imena host u IP adresu bili upućivani istom serveru koji bi postao usko grlo
 - Korišćenje centralizovanih algoritama nije poželjno u DS
 - Npr centralizovani algoritam za rutiranje
 - Treba koristiti samo decentralizovane algoritme

Skalabilnost (nast.)

- Decentralizovani algoritmi imaju sledeće osobine
 - Ni jedna mašina nema kompletnu informaciju o stanju sistema
 - Mašine donose odluku samo na osnovu lokalnih podataka
 - Otkaz jedne mašine ne narušava sistem
 - Nema pretpostavke o postojanju globalnog časovnika!
- Poslednja osobina je veoma važna (mada nije očigledna)
 - Npr. Bilo koji algoritam koji bi zahtevao sledeće: "Tačno u 12:00:00 mašine treba da provere veličinu svog reda čekanja" ne bi korektno radio jer je nemoguće postići tačnu sinhronizaciju časovnika u DS.
 - Distribuirani algoritmi moraju da uzmu u obzir da ne postoji tačna (egzaktna) sinhronizacija časovnika.
 - Što je veći sistem, to je neslaganje veće.

Skalabilnost (nast.)

* Geografska skalabilnost ima svoje probleme

- Jedan od problema je što je teško proširiti DS koji je bio projektovan u okviru LAN.
 - U LAN je komunikacija između procesa sinhrona
 - Kod sinhronne komunikacije, strana koja zahteva uslugu (klijent) se blokira dok čeka na odgovor od severa.
 - Ovakva komunikacija dobro funkcioniše u LAN jer je vreme čekanja kratko (nekoliko μs), dok je u WAN duže i za tri reda veličine (što je za interaktivne aplikacije dugo)

Tehnike skaliranja

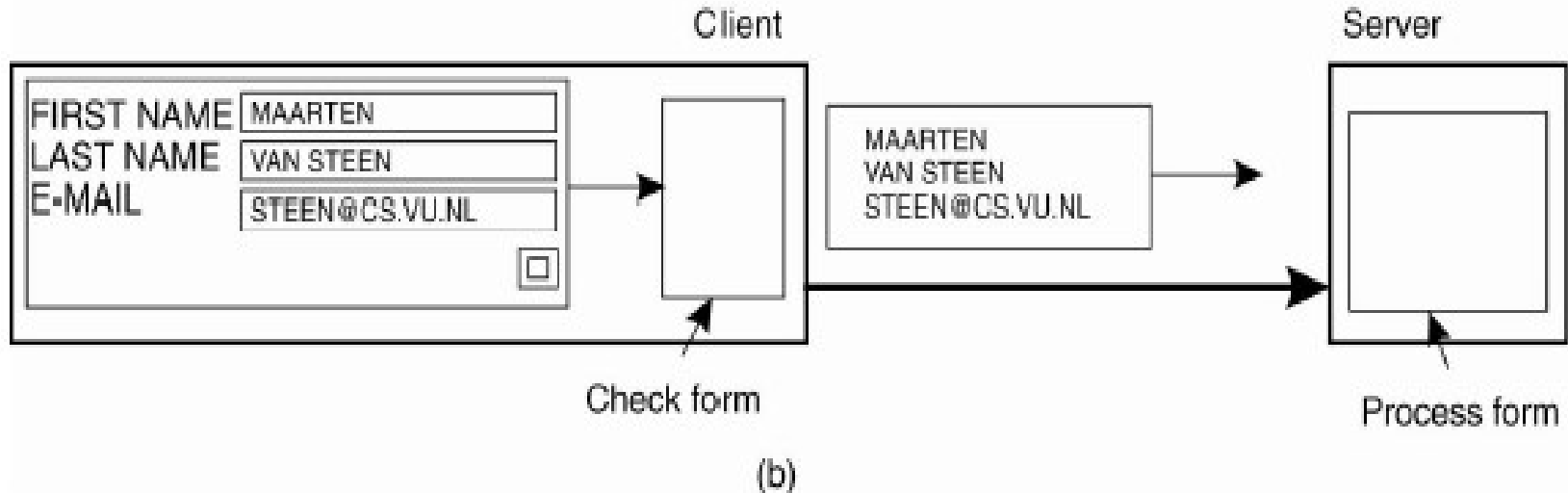
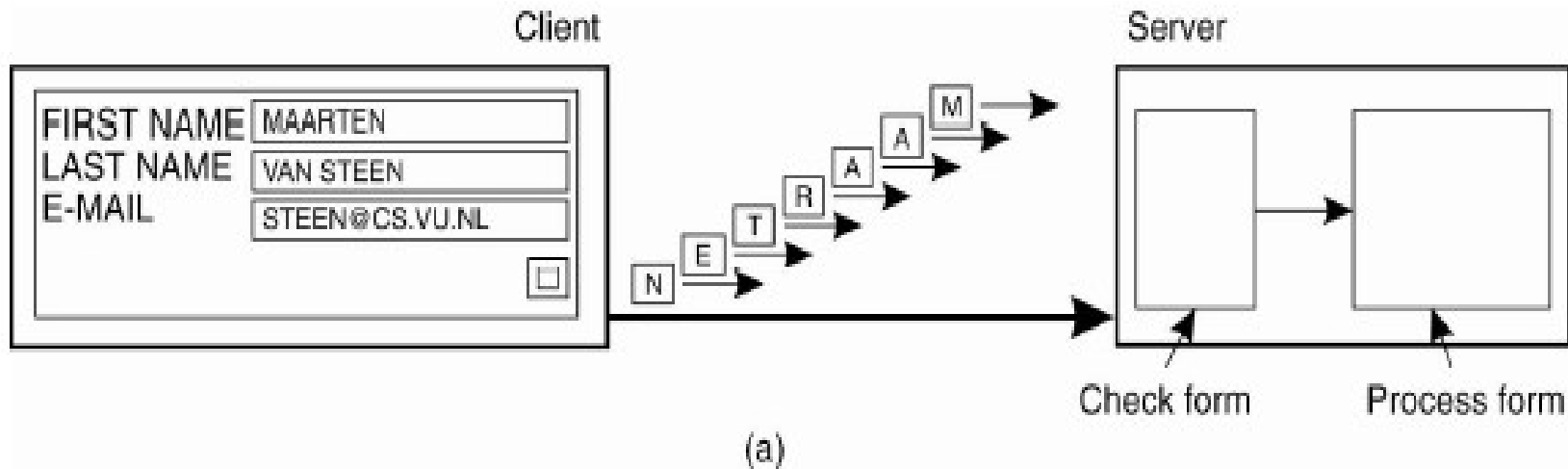
* Tri tehnike

- Skrivanje komunikacionog kašnjenja
- Distribucija
- Replikacija

* Skrivanje komunikacionog kašnjenja

- Koriste se asinhrona komunikacije umesto sinhronih
 - Kada se uputi zahtev udaljenom servisu, klijent se ne blokira dok čeka da stigne odgovor, već radi neki drugi posao
 - Kada stigne odgovor od servera, generiše se prekid koji obaveštava da je poruka stigla
- Download-ovati deo koda na klijent stranu da bi se ubrzala obrada
 - Asinhrona komunikacije nisu od koristi kod interaktivnih aplikacija
 - U takvim slučajevima je mnogo bolje rešenje smanjiti potrebe za komunikacijom, tako što se recimo, deo obrade koji se normalo izvršava na serverskoj strani, izvršava na strani klijenta (ovo se koristi kod weba u formi Java Script koda).
 - Tipičan primer gde ovaj prilaz ima smisla je pristup bazama podataka koje koriste forme.
 - » Uobičajeno je da se popunjavanje forme obavlja slanjem odvojene poruke za svako polje u formi, a zatim čeka potvrda od servera. (server recimo može obaviti sintaksnu proveru pre nego što prihvati upis)
 - » Mnogo bolje rešenje je da se klijentu uz formu isporuči i deo koda za popunjavanje forme (i eventualno proveru grešaka), a da klijent serveru vrati kompletno popunjenu formu

Tehnike skaliranja (nast.)



- a) Tradicionalni način procesiranja formi
- b) Procesiranje formi na strani klijenta

Tehnike skaliranja (nast.)

* Distribucija

- Podrazumeva da se komponente dele na manje delove, a zatim se ti delovi distribuiraju na više mašina u sistemu
 - Tipičan primer je organizacija DNS
 - Web je takođe primer korišćenja distribucije.
 - Korisniku web može izgledati kao jedinstveni sistem u kome se nalaze dokumenti koji su identifikovani svojim jedinstvenim imenima (URL)
 - Međutim, web je fizički distribuiran kroz veliki broj web servera
 - » Jedino zbog ovakve distribucije dokumenta web je u stanju da opsluži veliki broj korisnika.

* Replikacija

- Pomaže da se poveća dostupnost i balansira opterećenje u sistemu da bi se postigle bolje performanse
 - U sistemima koji su geografski distribuirani poželjno je da postoji kopija resursa blizu mesta korišćenja kako bi se smanjilo komunikaciono kašnjenje.
 - Postojanje replika može da dovede do problema konzistentnosti (npr. modifikovanje jedne kopije)

* tehnike skaliranja mogu prouzrokovati probleme

- postojanje više kopija (u kešu ili repliciranih) dovodi do problema konzistencije – modifikacija jedne kopije dovodi do neslaganja sa ostalim kopijama.
- da bi se kopije stalno održavale konzistentnim potrebna je globalna sinhronizacija
- globalnu sinhronizaciju je gotovo nemoguće postići u distribuiranom sistemu (ili je neisplativo sa stanovišta performansi)

* DS može tolerisati izvestan nivo nekonzistentnosti!

Šta je middleware

✱ Komunikacija između procesa je u srcu svih distribuiranih sistema.

- Komunikacija u DS je uvek bazirana na slanju poruka, tj. ne postoji zajednički adresni prostor.
- Da bi se pojednostavilo pisanje i razvoj distribuiranih aplikacija potrebno je obezbediti softversku podršku koja će aplikativnog programera osloboditi detalja vezanih za interprocesnu komunikaciju, sinhronizaciju, bezbednost itd.

✱ U OSI referentnom modelu, poslednji nivo je aplikativni.

- Ovaj nivo je originalno trebalo podrška za standardne mrežne aplikacije kao što su e-mail, prenos fajlova, emulaciju terminala i sl.

* Ono što OSI referentnom modelu nedostaje je jasna razlika (granica) između aplikacije, aplikativno-specifičnih protokola i protokola opšte namene (general purpose protocols).

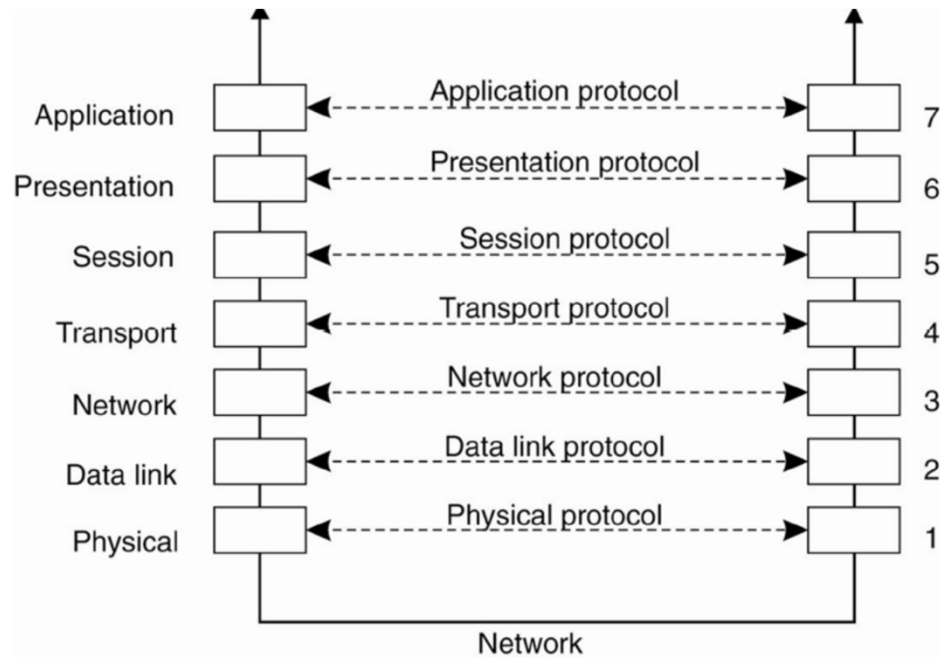
- Npr. Internetov FTP protokol je protokol za razmenu fajlova između klijenta i servera.
 - Ovaj protokol ne treba poistovećivati sa *ftp* programom koji predstavlja korisničku aplikaciju za prenos fajlova.
- Drugi primer aplikativno specifičnog protokola je HTTP koji omogućava prenos web stranica.
 - Ovaj protokol je implementiran u aplikacijama kao što su web browser-i i web serveri.

✱ Postoji i mnogo protokola opšte namene koji su od koristi za mnoge aplikacije, a ne mogu se kvalifikovati kao transportni protokoli.

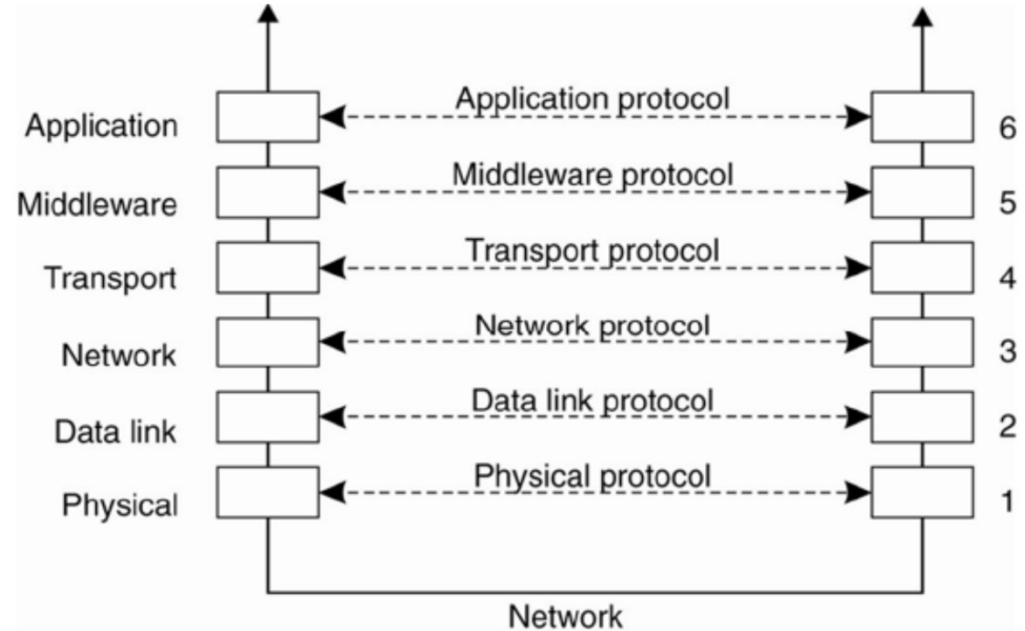
- Ti protokoli spadaju u kategoriju *middleware* protokola.
 - Ti protokoli pružaju različite usluge aplikaciji, tj. aplikativnom nivou.
- Npr. jedna od usluga koju ovi protokoli pružaju je usluga autentifikacije (provere identiteta) i autorizacije (prava pristupa) koje se zahtevaju u mnogim aplikacijama.
 - Zbog toga ima smisla ovu uslugu integrisati u middleware sistem kao opštu slugu.
- Drugi primer usluge koja se često zahteva je sinhronizacija.
 - Pri tome pod sinhronizacijom podrazumevamo neku vrstu vremenske sinhronizacije (da se utvrdi redosled događaja u DS) ili sinhronizaciju kod pristupa zajedničkom deljivom resursu (uzajamno isključivanje).
 - Ta usluga je nezavisna od aplikacije i ima je smisla implementirati kao uslugu koju mogu koristiti različite aplikacije

- Kada je reč o komunikaciji, videli smo da transportni protokoli TCP i UDP omogućavaju razmenu poruka između distribuiranih procesa.
 - Međutim, ta usluga je implementirana na niskom nivou i zahteva dosta napora pri programiranju distribuiranih aplikacija.
 - Npr. da bi aplikacija pristupila transportnim uslugama programer koristi skup API-ja koje mu nudi lokalni OS: kreiranje i otvaranje socket-a, kreiranje bafera, upis u bafer, čitanje iz bafera itd.
 - Middleware komunikacioni protokoli pružaju viši nivo komunikacionih usluga i oslobađaju aplikativnog programera detalja vezanih za komunikaciju između procesa.
 - Komunikacija je skrivena iza poziva procedure (remote procedure call - RPC) ili metoda (remote method invocation - RMI) ili je na raspolaganju skup funkcija za razmenu poruka (message oriented).

Izmenjeni OSI referentni model



Originalni OSI referentni model

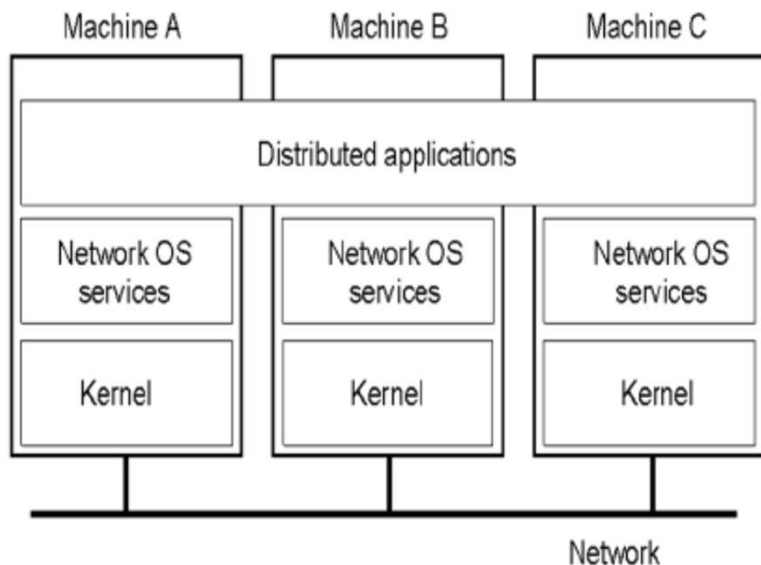


Modifikovani OSI model

Middleware

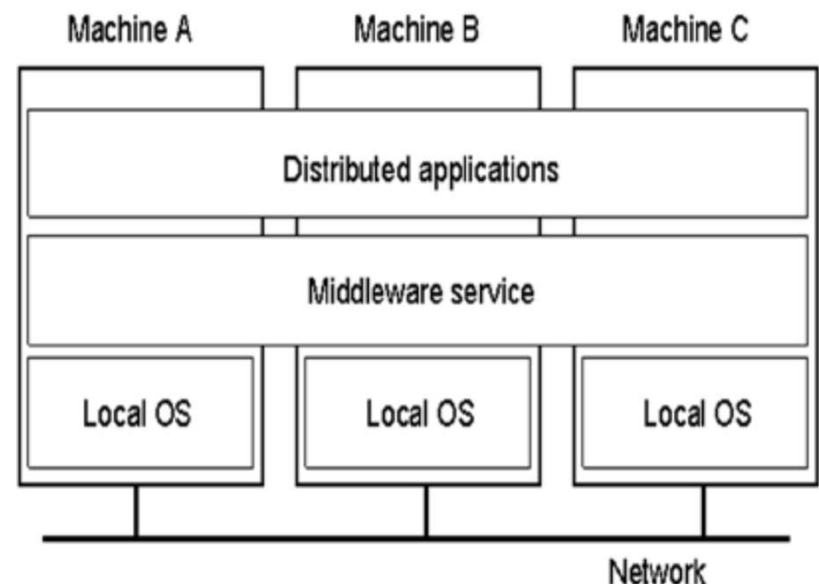
* Platforma na kojoj su izgrađeni DS su računarske mreže

- Mrežni OS pruža mogućnost korisnicima da pristupe uslugama koje su locirane na nekoj drugoj mašini (implementira npr. TCP/IP protokol stek)
- Da bi se pojednostavilo pisanje distribuiranih aplikacija i njihova integracija u DS na Mrežni OS je moguće je dodati još jedan softverski sloj (MIDDLEWARE) sa ciljem da sakrije heterogenost platforme na kojoj je sistem izgrađen od aplikacije, i da se sakrije komunikacija, tj. distribuciju.



Network/OS based
distributed system

Middleware based
distributed system



Middleware (nast.)

- * Svaki lokalni OS obavlja upravljanje lokalnim resursima, pored toga što obezbeđuje komunikaciju sa drugim računarima
 - Middleware ne upravlja radom pojedinačnog čvora u mreži, to radi lokalni OS
 - Osnovni cilj middleware je da se sakrije heterogenost platforme na kojoj je sistem izgrađen od aplikacije.
 - Middleware sistemi nude kompletan skup usluga aplikaciji i ne dozvoljavaju korišćenje ničeg drugog do njihovih interfejsa prema uslugama
 - Nije moguće preskočiti Middleware nivo i direktno pozvati uslugu OS (npr. Koristiti sockete za komunikaciju sa drugim procesom)
 - Na žalost ne postoji jedan standard koji definiše kako izgledaju protokoli u middleware nivou.
 - Kada je reč o middleware *komunikacionim* protokolima postoji nekoliko modela middleware: RPC, RMI, message oriented (MPI).

RPC komunikacioni middleware model

- * Middleware model zasniva se na pozivu udaljenih procedura – RPC model (Remote Procedure Call). –
- * resursi se modeliraju kao funkcije (procedure)
 - U ovom modelu naglasak je stavljen na skrivanje mrežnih komunikacija tako što je omogućeno procesima da pozivaju procedure čija je implementacija locirana na drugoj mašini.
 - Kada se poziva takva procedura, parametri se transparentno prenose do udaljene mašine na kojoj se zatim procedura izvršava, a rezultat izvršenja šalje pozivaocu.
 - Zbog transparentnosti komunikacije, pozivajući proces ima utisak da se procedura izvršila lokalno, tj. nije "svestan" da je došlo do komunikacije kroz mrežu (osim možda što je osetio usporenje u radu)
- * Na ovaj način se postiže transparentnost u odnosu na lokaciju, implementaciju i jezik
- * Problemi
 - Kako preneti parametre
 - Povezivanje (pronalaženje udaljene procedure)
 - Semantika u slučaju grešaka

Modeli middleware (nast.)

* Objektno orijentisani pristup

* resursi se modeluju kao objekti koji sadrže skup podataka i funkcija nad podacima. Udaljenom resursu se pristupa kao objektu

- Logična posledica RPC modela: ako se pozivi procedura mogu realizovati van granica jedne mašine, onda je moguće pozvati i objekte koji su smešteni na udaljenim mašinama.
- Ovaj prilaz doveo je do pojave različitih middleware sistema koji podržavaju koncept distribuiranih objekata.
 - Suština distribuiranih objekata je da svaki objekt implementira interfeis koji skriva sve unutrašnje detalje objekta od korisnika.
 - Interfeis sadrži deklaraciju metoda koje objekat implementira
 - Jedino što proces vidi od objekta je njegov interfeis.
 - Distribuirani objekti su često implementirani tako da je svaki objekt lociran na jednoj mašini, a interfeis je raspoloživ na mnogim drugim mašinama.
 - Kada proces pozove metod, interfeis lociran na lokalnoj mašini jednostavno transformiše poziv metoda u poruku koja se prosleđuje objektu
 - Objekt izvršava traženi metod i vraća rezultat
 - Interfeis transformiše odgovor u vrednost koja se vraća koju zatim koristi pozivajući proces.
 - Kao i u slučaju RPC proces ne mora biti svestan mrežne komunikacije.

Middleware modeli



(RPC, RMI, MPI, CORBA - remote calls, object invocation, messages, ...)

(sockets, IP, TCP, UDP, ...)

Tipovi DS

* Na arhitekturnom nivou

- klijent-server
- peer-to-peer

* U odnosu na oblasti primene

● Distribuirani računarski sistemi

- Klasteri
- Grid

● Distribuirani informacioni sistemi

- Orjentisani ka poslovnim aplikacijama
 - Sistemi za obradu transakcija
 - Sistemi za integraciju poslovnih aplikacija

● Ugrađeni (sveprisutni) DS

- Mobilni računari, ugrađeni (embeded) računari, komunikacioni sistemi

Distribuirani računarski sistemi

* Koriste se za izvršenje visokoperformansnih zadataka (izračunavanja)

● Klasteri

- Klasteri se sastoje od grupe sličnih radnih stanica ili PC, koji se nalaze na malom rastojanju, povezani preko veoma brze lokalne mreže.
- Svaki čvor izvršava isti OS

● Grid

- Sastoji se od grupe (federacije) računarskih sistema, koji mogu biti u različitim administrativnim domenima, razlikovati se u pogledu hw, sw i mrežne tehnologije koju koriste

Klasteri

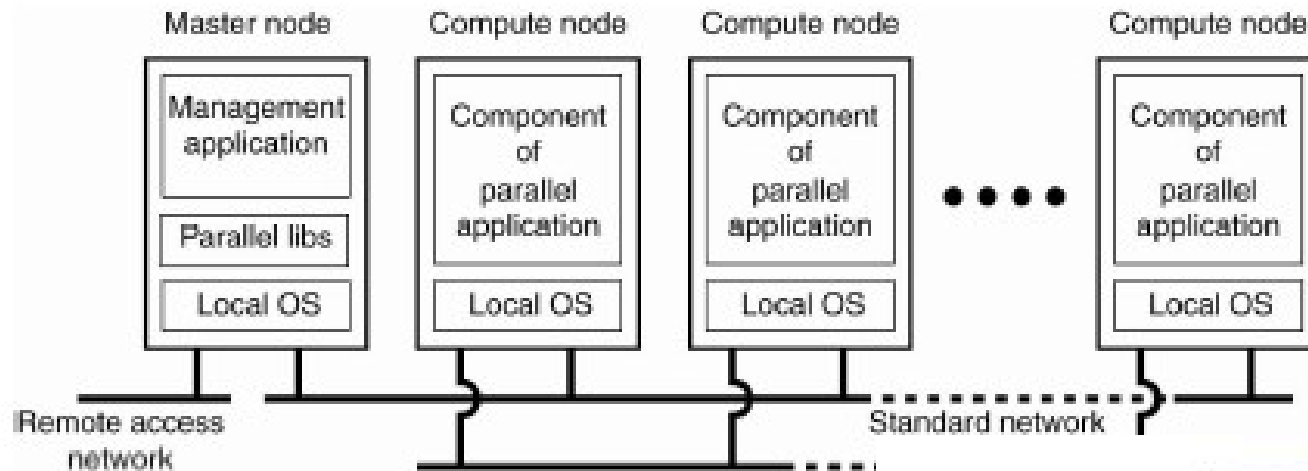
* Postali su popularni kada je odnos cena/performance PC i radnih stanica postao povoljan.

- Postalo je isplativo i tehnički atraktivno napraviti superračunar koristeći raspoloživu tehnologiju povezivanjem relativno jednostavnih računara brzom komunikacionom mrežom.
- Ključna osobina homogenost
 - Svi čvorovi imaju isti OS, svi su povezani istom mrežom
 - Klasteri se koriste za paralelno izračunavanje

* Beowulf

- Klaster baziran na Linux OS.
- Sastoji se od grupe računarskih čvorova kojima upravlja jedan master čvor
 - Master obavlja alokaciju zadataka čvorovima u klasteru, upravlja redom čekanja poslova, obezbeđuje interfeis sa korisnikom.
 - Master u suštini izvršava middleware potreban za izvršenje programa i upravljanje klasterom
 - Ostali čvorovi najčešće imaju samo standardni OS

Klasteri (nast.)



- * Middleware sadrži biblioteku funkcija za izvršenje paralelnih programa
 - Funkcije obezbeđuju efikasne mehanizme za razmenu poruka
- * MPI – Message Passing Interface
 - Najčepći tip middleware kod klastera



Grid

✱ Grid je distribuirani računarski sistem čije komponente mogu biti locirane na velikom geografskom području

- Grid sistemi su veoma heterogeni a integracija je postignuta preko zajedničke servisno-orjentisane softverske arhitekture (middleware) .
- Resursi iz različitih organizacija (i administrativnih domena)
 - Resursi su različite usluge, računari, klasteri, diskovi, baze podataka, senzori,...
- Korisnici grida su organizovani u tzv. "virtuelne organizacije"
 - Korisnici koji pripadaju istoj virtuelnoj organizaciji imaju pravo pristupa resursima koje poseduje virtuelna organizacija
- Middleware grid sistema treba da obezbedi pristup resursima iz različitih administrativnih domena i to samo onim osobama i aplikacijama koji pripadaju virtuelnoj organizaciji.
 - Neophodno je obezbediti mehanizme za autentifikaciju (proveru identiteta) korisnika ili aplikacije.

Distribuirani informacijski sistemi

Sistemi za obradu transakcija

- * Transakcija predstavlja skup operacija koje se obavljaju kao jedna nedeljiva (atomična) operacija.
 - Sistem za obradu transakcija obezbeđuje da sve ili ni jedna operacija u transakciji budu izvršene bez greške.
 - nakon obavljanja transakcije sistem (obično baze podataka ili fajl sistem) mora da ostane u poznatom, konzistentnom stanju tako što obezbeđuje da se operacija koje su međusobno zavisne ili sve izvrše ili se ni jedna ne izvrši.
 - Primeri sistema
 - podizanje novca putem platnih kartica (bankomat vam isporučuje novac i umanjuje vrednost na vašem računu – ili će se obe operacije obaviti ili ni jedna)
 - sistem za rezervaciju avio karata (avio kompanija potvrđuje rezervaciju i smanjuje broj slobodnih mesta, skida novac sa vaše platne kartice, (ponekad) povećava broj obroka za dati let,..)
 - transakcije se moraju procesirati uvek na isti način. Da bi se to postiglo sistemi za obradu transakcija se projektuju tako da pružaju isti interfejs za svaku transakciju bez obzira na korisnika.

tipične komande za rad sa transakcijama

* *Begin_transaction*

- početak transakcije

* *End_transaction*

- okončanje transakcije
 - između *Begin_transaction* i *End_transaction* se nalazi telo transakcije

* *Abort_transaction*

- okončanje transakcije i vraćanje na prethodne vrednosti

* *Read*

- čitanje podataka iz fajla, tabele,... za transakciju

* *Write*

- upis podataka u fajl, tabelu,.. za transakciju

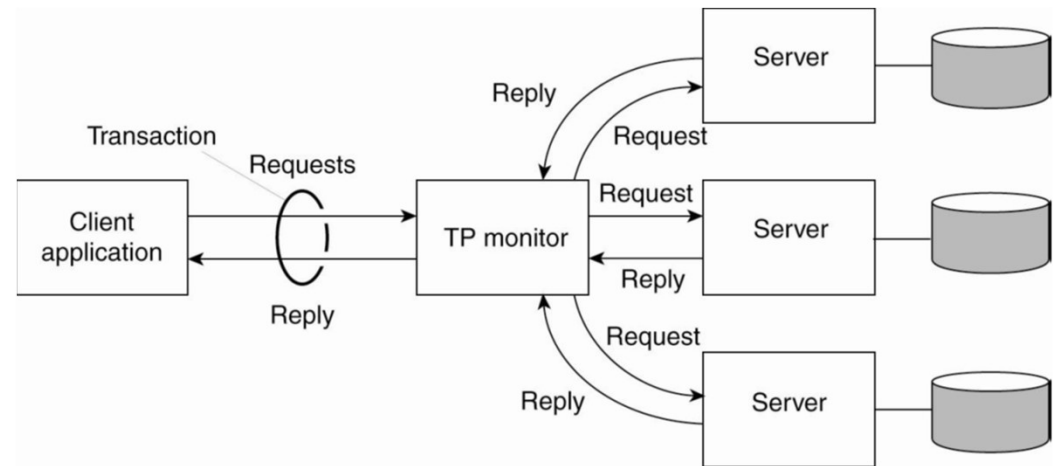
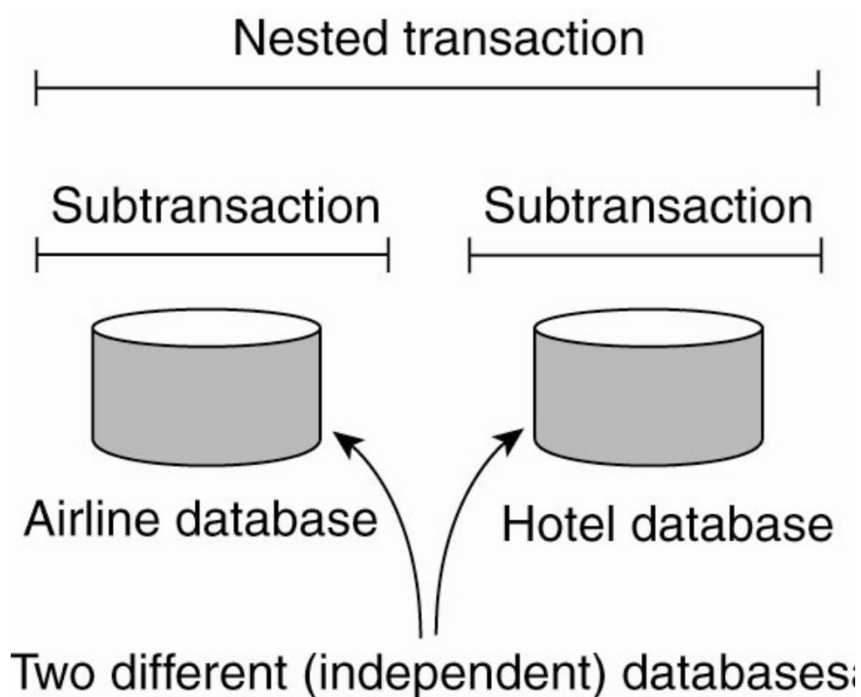
Osobine transakcija - ACID

- * Svaka transakcija mora da zadovolji ACID test pre nego što se dozvoli modifikacija sadržaja
- * A – Atomicity–
 - podrazumeva da se transakcija obavi kompletno ili se uopšte ne obavi
- * C - Consistency –
 - transakcija ne ugrožava skup invarijanti (ograničenja, konstanti) sistema.
 - Npr. ako je uslov da sve transakcije nad bazom podataka moraju imati pozitivnu vrednost, bilo koja transakcija sa negativnom vrednošću će biti odbijena. (npr. ako pokušate da podignete više novca nego što imate na računu)
- * I – Isolation –
 - transakcije su medjusobno izolovane, tj. dve konkurentne transakcije moraju biti serijalizovane (izvršavaju se u nekom nedeterminisanom redosledu, ali taj redosled mora biti vidljiv na isti način za sve transakcije)
- * D - Durability –
 - kada se transakcija obavi ne može se poništiti!

Distribuirana transakcija

* U DS transakcija je obično sastavljena od više podtransakcija koje su raspoređene na više servera.

- Monitor za obradu transakcija (TP monitor) je middleware koji omogućava aplikaciji da obavi pristup više servera/baza podataka nudeći joj transakcioni programski model. TP monitor koordinira potvrdu podtransakcija prema standardnom protokolu.



Sveprisutni Distribuirani sistemi

- * Prva dva tipa DS se karakterišu relativnom visokom stabilnošću:
 - čvorovi su fiksni i imaju uglavnom stalnu i kvalitetnu mrežnu konekciju
- * Sa pojavom mobilnih i ugrađenih (embedded) računarskih sistema, stvari se menjaju: nestabilnost je uobičajeno ponašanje.
 - Uređaji u ovim DS su uglavnom mali, napajaju se pomoću baterija, mobilni su, imaju samo bežične veze.
 - Važna osobina ovakvih DS je odsustvo administrativnog upravljanja
 - Uređaji sami moraju da otkriju okruženje i ugnjezde se u njega
 - Uređaji moraju biti svesni da se okruženje stalno može menjati
- * Primeri
 - Kućni sistemi
 - Organizovani oko jednog centralnog računara (automatsko paljenje svetla, sistemi za navodnjavanje, alarmni sistemi, različiti kućni aparati...)
 - Elektronski sistemi za monitoring pacijenata
 - Senzorske mreže – sistemi za akviziciju podataka i nadzor
 - Merenje temperature, vlažnosti,... i slanje informacija do bazne stanice gde se obavlja procesiranje